



Feast9

Dental Practice Management System

Deployment Handover Guide

Confidential — for the deploying firm only

Document date	August 2026
Application	Feast9 — Dental Practice Management System
Version	Final (inclusive of DPDP Phase 1 + Phase 2)
Prepared by	Owner / Development team
Intended audience	Deployment / DevOps firm

This document contains environment credentials and configuration instructions. Do not share beyond the deployment team. Discard or securely archive once live deployment is confirmed.

1. Application Overview

Feast9 is a single-practitioner dental clinic management system built on **Python / Flask** with a **SQLite** database. It handles patients, treatment cases, appointments, billing, consent forms, prescriptions, reports, analytics, and DPDP Act 2023 compliance (Phases 1 and 2). There is no separate frontend build step — the application is a single Python package served by Gunicorn behind a reverse proxy.

Language / framework	Python 3.10+ / Flask 3.x
Database	SQLite 3 (single file — no external DB server needed)
WSGI server	Gunicorn
Reverse proxy / TLS	Caddy (recommended) or Nginx + Certbot
Containerisation	Docker + Docker Compose (included in package)
Expected traffic	Single practitioner, low-concurrency — 1–2 workers sufficient
Persistent storage	One directory (DATA_DIR) for the database, uploads, backups, branding

2. Infrastructure Requirements

2.1 Minimum server specification

CPU	1 vCPU (2 recommended for comfort)
RAM	512 MB minimum — 1 GB recommended
Disk	10 GB minimum SSD. DATA_DIR grows ~5–20 MB/year for a single practice; PDF/signature uploads add more. 20 GB is comfortable for 5 years.
OS	Ubuntu 22.04 LTS or 24.04 LTS (recommended). Debian 12 also works.
Network	Public IPv4. Port 443 (HTTPS) and 80 (HTTP → redirect). Outbound internet for package installation only.

Recommended hosting options (in cost order):

- **Hetzner Cloud CX22** — ~\$400/month, 2 vCPU / 4 GB RAM, 40 GB SSD, Singapore or EU region. Best price/reliability ratio.
- **DigitalOcean Droplet (Basic, 1 GB)** — ~\$600/month.
- **Render Starter** — ~\$600/month, managed platform, least server management.
- **Oracle Cloud Always Free** — \$0, but signup can be difficult from India and there is no SLA.

2.2 Software prerequisites (server)

- Docker Engine 24+ and Docker Compose v2 plugin
- Git (to pull the application)
- Caddy 2 or Nginx + Certbot (for HTTPS termination)
- An SSH key-pair for server access

All Python dependencies (Flask, Gunicorn, openpyxl, reportlab, Pillow) are installed inside the Docker image at build time. Nothing extra needs to be installed on the host beyond Docker.

2.3 Domain and DNS

- A registered domain or subdomain pointing to the server's public IP (A record, TTL 300 or lower for quick propagation).
- TLS/SSL certificate — auto-provisioned by Caddy via Let's Encrypt (no manual certificate management needed).

3. Delivered Package Contents

The handover zip file (**feast9_web.zip**) contains:

- **app/** — Flask application package (routes, templates, static files, PDF generation, Excel export, backup module)
- **Dockerfile** — builds the production container image
- **docker-compose.yml** — orchestrates the container + persistent volume
- **render.yaml** — one-click deploy config for Render.com (alternative)

- **.env.example** — template for all required environment variables
- **wsgi.py** — Gunicorn entry point
- **requirements.txt** — pinned Python dependency versions
- **seed_demo_data.py** — optional: populates the system with sample data for testing. **Do not run this on the live instance.**
- **reset_admin_password.py** — emergency CLI password reset script
- **README.md** — full technical reference

4. Environment Variables

These must be set before the container starts. Never commit them to version control. Store them in the server's .env file (outside the repo) or in your hosting platform's secrets manager.

SECRET_KEY	A long, random string used to sign session cookies. Generate with: <code>python3 -c "import secrets; print(secrets.token_hex(32))"</code> — REQUIRED in production.
DATA_DIR	Absolute path to the persistent data directory on the host (mounted into the container). Defaults to /data inside the container. Example: /var/lib/feast9
SESSION_COOKIE_SECURE	Set to 1 once HTTPS is live. Prevents session cookies being sent over plain HTTP.
FLASK_DEBUG	Must be 0 (or absent) in production. Never set to 1 on a live server.
PORT	Internal port Gunicorn listens on. Defaults to 8000. The reverse proxy connects to this.

Example .env file:

```
SECRET_KEY=replace_this_with_a_real_64_char_random_string
DATA_DIR=/var/lib/feast9
SESSION_COOKIE_SECURE=1
FLASK_DEBUG=0
PORT=8000
```

5. Deployment Steps

5.1 Docker deployment (recommended)

```
# 1. SSH into the server
ssh ubuntu@your-server-ip

# 2. Install Docker (Ubuntu)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER && newgrp docker

# 3. Upload and extract the application
scp feast9_web.zip ubuntu@your-server-ip:/opt/
cd /opt && unzip feast9_web.zip && cd feast9_web

# 4. Create the data directory
sudo mkdir -p /var/lib/feast9
sudo chown $USER:$USER /var/lib/feast9

# 5. Create the environment file
cp .env.example .env
nano .env # fill in SECRET_KEY and other values

# 6. Build and start
docker compose --env-file .env up -d --build

# 7. Verify it's running
docker compose ps
curl -I http://localhost:8000
```

5.2 HTTPS with Caddy (recommended)

Install Caddy, then create `/etc/caddy/Caddyfile`:

```
your-domain.com {
  reverse_proxy localhost:8000
}
```

```
sudo apt install -y caddy
sudo systemctl enable --now caddy
# Caddy auto-provisions a Let's Encrypt TLS certificate
```

5.3 Render.com one-click deployment (alternative)

- Push the `feast9_web` folder to a private GitHub repository.
- In Render: New → Blueprint. Point it at the repo. Render reads `render.yaml` and provisions the service automatically.
- Under the service's Environment settings, add `SECRET_KEY`, `DATA_DIR=/data`, and `SESSION_COOKIE_SECURE=1`.
- Under Disks, add a 1 GB disk mounted at `/data` — **this is essential**; without it the database is lost on every redeploy.

6. First-Run Configuration (hand off to clinic)

Once the server is live, the clinic owner completes this one-time setup by visiting the application URL in a browser:

- 1 **Create admin account** — the first visit redirects to a setup page. Choose a username, a strong password, and a security question (used for self-service password recovery — no email server needed).
- 2 **Enter clinic details** — Settings → Clinic Details: name, address, phone, email. These appear on every printed prescription and PDF.
- 3 **Upload clinic logo** — Settings → Clinic Logo. PNG or JPG, auto-resized.
- 4 **Add doctors** — Doctors tab. Each doctor gets a distinct calendar colour for the appointments view.
- 5 **Review case types** — Case Types tab. A standard dental procedure list is pre-loaded; add or deactivate as needed.
- 6 **Historic data import** (optional) — Import Data tab. Download the Excel template, fill patient records, upload. Up to 8,000 rows processed in a single upload.

7. Persistent Data — Critical

Everything lives in DATA_DIR. If this directory is lost (container reset without a mounted volume, disk failure, accidental deletion), ALL data is lost. This is the single most important infrastructure decision.

Database file	DATA_DIR/feast9.db — the entire application database
Uploaded files	DATA_DIR/uploads/ — consent signature images, scanned consent PDFs, branding logo
Backups	DATA_DIR/backups/ — automatic daily snapshots + manual downloads (14-day rotation)
Branding	DATA_DIR/branding/ — custom logo if uploaded

Recommended backup strategy:

- The app creates an automatic daily SQLite snapshot in DATA_DIR/backups/. This protects against accidental in-app deletion.
- Set up a nightly cron job to copy DATA_DIR to off-server storage (S3/R2/Backblaze). The included **backup.sh** script creates a timestamped .tar.gz of the full DATA_DIR.
- The clinic owner can also use **Settings** → **Download Backup Now** and **Export to Excel** at any time for their own copies.

Example cron (nightly at 2 AM):

```
0 2 * * * /opt/feast9_web/backup.sh >> /var/log/feast9_backup.log 2>&1
```

8. Pre-Launch Security Checklist

Verify each item before handing the live URL to the clinic:

- ✓ SECRET_KEY is a real random string (not the .env.example placeholder)
- ✓ SESSION_COOKIE_SECURE=1 is set (requires HTTPS to be live first)
- ✓ FLASK_DEBUG=0 (never 1 in production)

- ✓ HTTPS is working — browser shows padlock, certificate is valid
- ✓ HTTP redirects to HTTPS (Caddy does this automatically)
- ✓ DATA_DIR is on a persistent mounted volume (test: restart container, verify data survives)
- ✓ Firewall: only ports 22, 80, 443 open (close 8000 from public internet)
- ✓ First-run admin account created with a strong password
- ✓ Security question set during first-run setup

9. Updating the Application

The database schema uses **additive-safe migrations** — `init_db()` runs `CREATE TABLE IF NOT EXISTS` on every boot, so pulling an update never wipes existing data. New columns are added automatically via the migration helpers.

To deploy an updated version:

```
# Upload new zip, extract, rebuild
scp feast9_web_vX.zip ubuntu@server:/opt/
cd /opt && unzip -o feast9_web_vX.zip
cd feast9_web
docker compose down
docker compose --env-file .env up -d --build

# The DATA_DIR volume is preserved – no data is lost
```

10. Emergency Recovery Procedures

10.1 Password reset (forgot login credentials)

The clinic owner can use **Forgot password?** on the login page if they set up a security question (Step 6 above). If that also fails:

```
# SSH into the server and run inside the container
docker exec -it feast9_web-web-1 python reset_admin_password.py
```

10.2 Restore from a backup

```
# Stop the app
docker compose down

# Replace the live database with the backup
cp /path/to/backup/feast9_YYYY-MM-DD.db /var/lib/feast9/feast9.db

# Restart
docker compose --env-file .env up -d
```

11. Support and Handover Contacts

Application owner	Feast9 Dental Clinic — contact details to be filled by owner
Deployment firm	To be filled by deploying firm
Post-deployment SLA	To be agreed between owner and deploying firm
DPDP Data Grievance	Settings → Security → Data Grievance Contact (must be configured by clinic owner on first login, per DPDP Act Section 13)

This document is issued alongside feast9_web.zip. Full technical documentation is in README.md inside the package.